

Toolchain Summary

Yufeng (Alex) Xia · yufeng.xia@ed.ac.uk · University of Edinburgh

Each entry below states what I have actually built or measured with the tool, not merely exposure to it. Proficiency is graded as *primary* (I have owned production or artifact-quality components with it), *working* (I have used it to obtain results that appear in a paper), and *familiar* (I have used it, but would not yet claim depth).

LANGUAGES

- **C++ (C++17)** *primary*
My main language. I own the inter-device transport data plane of *Morphling* (EdgeSys'26 Best Paper artifact): event-driven proxy client/server, connection layer, memory pooling — around 2,400 lines in an 18k-line C++/CUDA codebase, built with CMake and covered by the project's CI. Comfortable with RAII and ownership discipline, `std::shared_ptr`-based buffer lifetime across asynchronous callbacks, lock-granularity decisions in a multi-threaded event loop, and reasoning about copies on a hot path.
- **C** *working*
Compiler and embedded work: the CoSenseAQ benchmark suite and cross-compiled targets for ARM Cortex-M0 and Cortex-A53; reading and instrumenting a large C codebase (OpenAirInterface) to add measurement hooks.
- **Python** *primary*
Experiment orchestration, measurement pipelines and figure generation across every project — large-scale deployment scripts, trace analysis, and the plotting toolchain for my papers and thesis (matplotlib, publication-quality figures).
- **Java** *working*
Production backend development during an industry internship: Spring Boot services, a live migration of a delivery platform from JFinal, code review and release testing.
- **Bash** *primary*
Deployment, orchestration and benchmarking automation, including multi-node cloud experiment drivers.

GPU AND ACCELERATOR TOOLCHAIN

- **CUDA (runtime and driver API)** *primary*
Pinned host memory (`cudaHostAlloc`, `cudaHostRegister`) with a bucketed pool of my own design; stream and event based timing; launching PTX/cubin kernels directly through the driver API in my MLIR compiler project.
- **GPU sharing mechanisms** *working*
SM-level partitioning via CUDA green contexts and MPS, used to co-locate a training workload with a latency-critical tenant on one device; measured the reconfiguration overheads that make one mechanism preferable to the other.
- **Profiling and performance analysis** *working*
Nsight-based kernel profiling, arithmetic-intensity and roofline analysis to establish whether a kernel is compute- or memory-bandwidth-bound — the analysis underpinning the co-location argument in *Weaver*. Also SM occupancy, memory-throughput and interference characterisation between co-resident workloads.
- **cuBLAS / cuBLASLt** *working*
GEMM execution paths in the Morphling worker pool, including multi-GPU dispatch and correctness checking against a CPU (MKL) reference.
- **PyTorch** *working*
Distributed foundation-model training runs at multi-site scale; intercepting backend dispatch and hooking autograd for emulation and scheduling purposes.

NETWORKING AND DATA-PLANE TOOLCHAIN

- **libevent / event-driven asynchronous I/O** *primary*
The Morphling transport is built on it: non-blocking connection handling, event-loop threads, buffered reads and writes, zero-copy scatter-gather via `evbuffer_add_reference` with shared-pointer cleanup. I also diagnosed and fixed connection-cleanup and device-failure deadlocks that only appeared at scale.
- **Serialisation frameworks** *working*
Protocol Buffers and FlatBuffers, benchmarked against each other along the full send path (serialise / size / transmit) to choose the framing and batching strategy; wrote the analysis up as design documents in the repository.
- **Measurement and diagnosis** *working*
iperf3 throughput comparison against real hardware, per-slot completion-time distributions, packet and timeline tracing, socket-state diagnosis (`netstat`, connection lifecycle) — used both to validate emulator fidelity and to find production-style bugs.
- **5G RAN stacks** *working*
OpenAirInterface and Open5GS: building and running a live 5G stack, instrumenting the MAC scheduler for deadline measurement, and driving traffic through it for experiments.
- **RDMA / SmartNIC / programmable switches** *familiar — to learn*
I have not yet worked with RDMA verbs, SmartNICs or P4. I state this plainly: my data-plane experience is at the host software layer, and moving it down into the network is precisely what attracts me to iSING Lab.

DISTRIBUTED SYSTEMS AND CLOUD

- **Microsoft Azure / CloudLab** *working*
Deployed and operated a distributed emulator across 130 cloud instances and 6,400 CPU cores for the *EmuRAN* evaluation, including provisioning, orchestration, cost management and failure recovery at that scale.
- **Docker and Kubernetes** *working*
Containerised reproducible research artifacts (including the Morphling image with the NVIDIA Container Toolkit) and multi-node deployment.
- **Distributed time and coordination** *working*
Implemented virtual-clock synchronisation keeping event-driven virtual time consistent across emulated devices, and the device-state tracking used for scheduling decisions.

COMPILER TOOLCHAIN

- **LLVM** *primary*
IR-level pass development for *CoSenseAQ* (first-author manuscript): variable-range analysis, type shrinkage and automatic quantization driven by sensor specifications, evaluated on embedded ARM targets for power, performance and area.
- **MLIR** *working*
Sole author of *TensorOps*, a multi-target tensor compiler: a custom dialect, operator fusion (matmul+relu) at the dialect level, lowering through Linalg to LLVM for CPU JIT and to NVVM/ROCDL for GPU, ending in PTX/cubin launched via the CUDA Driver API.
- **Cross-compilation and embedded toolchains** *working*
ARM Cortex-M0 and Cortex-A53 toolchains, binary size and cycle-level measurement on real hardware.

- **Git**

primary

Daily use in shared repositories: feature branches, pull requests and review (my Morphling contributions landed through reviewed PRs), rebasing, bisecting to locate regressions.

- **Build and CI**

working

CMake for mixed C++/CUDA projects, GitHub Actions (build, format, CodeQL, protobuf breaking-change and numerical-consistency checks), pre-commit hooks, clang-format.

- **Documentation**

primary

I write design and analysis documents as a matter of habit — the Morphling repository contains a series I authored on serialisation cost, virtual-time synchronisation, connection cleanup and failure-mode analysis. LaTeX for papers and figures throughout.